

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Design a student system in Python with:

Address class (street, city, zipCode)

Student class with name, age, Address, and course list

Store age as a protected attribute and control it using @property

Methods: add_course() and display()

Extend it with:

ScholarshipStudent (add scholarshipAmount and override display())

Your implementation should clearly show:

Composition (Student HAS-A Address) Proper data validation using @property (age must be valid)

Inheritance and overriding (use super() in display)

Understanding of mutable behavior (course list updates persist)

Answer:

```
class Address:
```

```
    def __init__(self, street, city, zipCode):
```

```
        self.street = street
```

```
        self.city = city
```

```
        self.zipCode = zipCode
```

```
    def display(self):
```

```
        return f"{self.street}, {self.city}, {self.zipCode}"
```

```
class Student:
```

```
def __init__(self, name, age, address, courses=None):
    self.name = name
    self._age = 0
    self.age = age
    self.address = address
    self.courses = courses if courses is not None else []
```

```
@property
def age(self):
    return self._age
```

```
@age.setter
def age(self, value):
    if value <= 0 or value > 120:
        raise ValueError("Invalid age")
    self._age = value
```

```
def add_course(self, course):
    self.courses.append(course)
```

```
def display(self):
    print("Name:", self.name)
    print("Age:", self.age)
    print("Address:", self.address.display())
    print("Courses:", self.courses)
```

```
class ScholarshipStudent(Student):
    def __init__(self, name, age, address, courses, scholarshipAmount):
        super().__init__(name, age, address, courses)
        self.scholarshipAmount = scholarshipAmount
```

```
def display(self):  
    super().display()  
    print("Scholarship Amount:", self.scholarshipAmount)
```

```
addr1 = Address("MG Road", "Tezpur", "784001")
```

```
s1 = Student("Gourisankar", 20, addr1)
```

```
s1.add_course("Python")
```

```
s1.add_course("Data Structures")
```

```
s1.display()
```

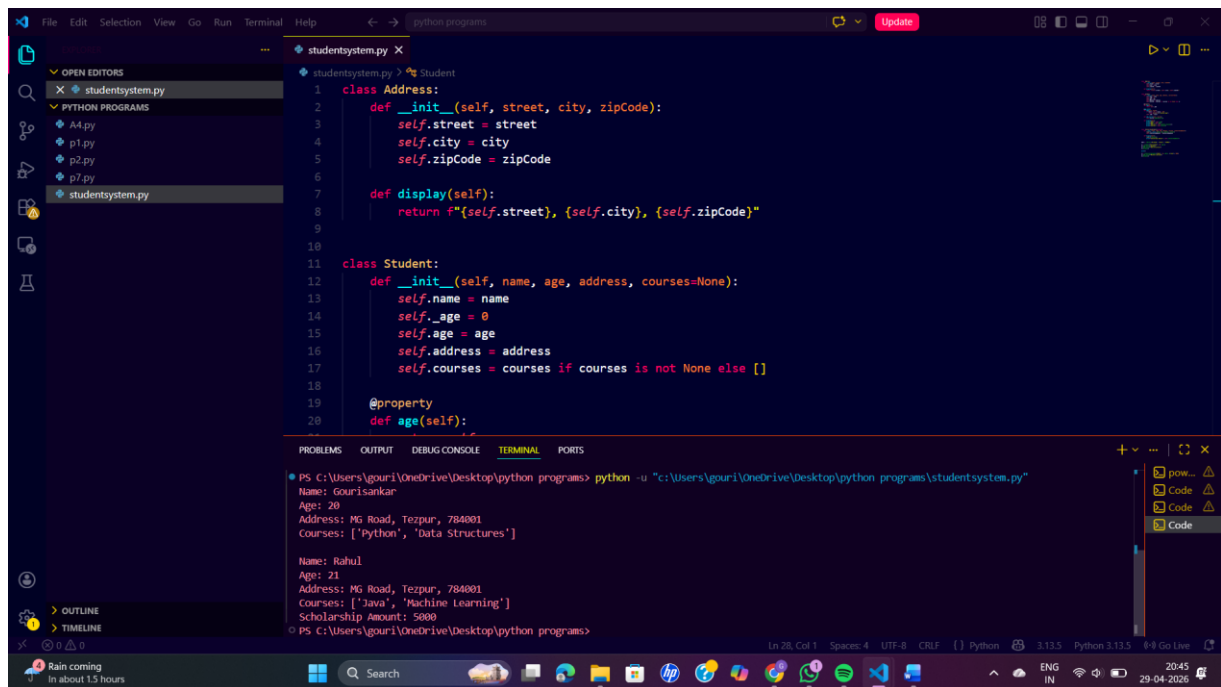
```
print()
```

```
s2 = ScholarshipStudent("Rahul", 21, addr1, ["Java"], 5000)
```

```
s2.add_course("Machine Learning")
```

```
s2.display()
```

Screenshots of the output:



The screenshot shows a Visual Studio Code editor window with a Python file named `studentsystem.py` open. The file contains two classes: `Address` and `Student`. The `Address` class has an `__init__` method that takes `street`, `city`, and `zipCode` as arguments and a `display` method that returns a string representation of the address. The `Student` class has an `__init__` method that takes `name`, `age`, `address`, and `courses` as arguments, and a `age` property that returns the age. The terminal output shows the execution of the program, which creates two `Student` objects and prints their details.

```
class Address:
    def __init__(self, street, city, zipCode):
        self.street = street
        self.city = city
        self.zipCode = zipCode

    def display(self):
        return f"{self.street}, {self.city}, {self.zipCode}"

class Student:
    def __init__(self, name, age, address, courses=None):
        self.name = name
        self._age = age
        self.age = age
        self.address = address
        self.courses = courses if courses is not None else []

    @property
    def age(self):
```

Terminal Output:

```
PS C:\Users\gouri\OneDrive\Desktop\python programs> python -u "C:\Users\gouri\OneDrive\Desktop\python programs\studentsystem.py"
Names: Gourisankar
Age: 20
Address: MG Road, Tozpur, 784001
Courses: ['Python', 'Data Structures']

Name: Rahul
Age: 21
Address: MG Road, Tozpur, 784001
Courses: ['Java', 'Machine Learning']
Scholarship Amount: 5000
PS C:\Users\gouri\OneDrive\Desktop\python programs>
```